European Union Cyber Resilience Act (CRA)

Project Phase #1

Due September 13

Q1.1 Scope (Richard)

(2+ stakeholders, 2-3 sentences each): Identify who must comply with the regulation, what activities or information are regulated, and which part(s) of the regulation your team is studying.

The main parties affected by these regulations (stakeholders) would be the manufacturers of the digital products and the security and compliance teams working within those manufacturers. We will be focusing on the post-release vulnerability handling and the processes that are used to investigate, remediate, document, and report vulnerabilities. Essentially our focus is on a vulnerability-management system that helps manufacturers identify security issues in products that have already been released and track the process to resolve them.

Q1.2 Key obligations (Richard)

Identify at least two software-relevant obligations. For each, provide:

- *The source of the obligation*
- *Required action and applicable conditions*
- *Evidence that would demonstrate compliance*
- *The consequences of failing to meet the selected obligation*

Handling Vulnerabilities: Under Article 13 and Annex I part II of the CRA, manufacturers are required to continue monitoring and handling vulnerabilities in their products throughout the product's support period. This would include identifying and documenting vulnerabilities and software components, investigating security issues, fixing vulnerabilities, and providing security updates when needed. Evidence of compliance

would include things such as vulnerability records, testing results, remediation records and records of security updates that were released (basically just document everything that is done).

Vulnerability and Incident Reporting: Under article 14 of the CRA, manufacturers must report actively exploited vulnerabilities and severe security incidents affecting their products. An initial warning for any actively exploited vulnerability must generally be submitted within 24 hours of becoming aware of it and after 72 hours, there must be a more detailed report submitted and a final report when a corrective or mitigative measure becomes available. Evidence of our compliance would include things such as timestamps showing when the vulnerability was discovered and reports submitted, investigation records, remediation information, and confirmation that our required reports were submitted on time.

Q1.3 Ambiguity (Richard)

(3-4 sentences): Identify one place where the regulation leaves room for interpretation or engineering judgment, and explain its effect on the project your team may decide to pursue.

In the CRA, they leave room for engineering judgement in it's requirement that manufacturers must address and remediate vulnerabilities "without delay." The judgement part is that the regulation does not provide a specific timeline that is acceptable for fixing every vulnerability, meaning it is up to the manufacturers to consider factors such as severity, exploitability, and the risk posed to users in determining when to address an issue. This can affect our project because the software could prioritize vulnerabilities and track how long they remained unresolved but should not determine automatically whether the manufacturer has violated the regulation. For example, if the vulnerability is low-severity and is difficult to reach in the product the timeline will probably be longer than one that is high-severity and easy to reach in the product but that doesn't mean the manufacturer has violated the regulation by not fixing it in time. Cases that exceed company-defined thresholds or involve uncertain risks could instead be flagged for review by a security or compliance team rather than just automatically determined by the program.

Q2.1 Workflow and boundary models (Richard)

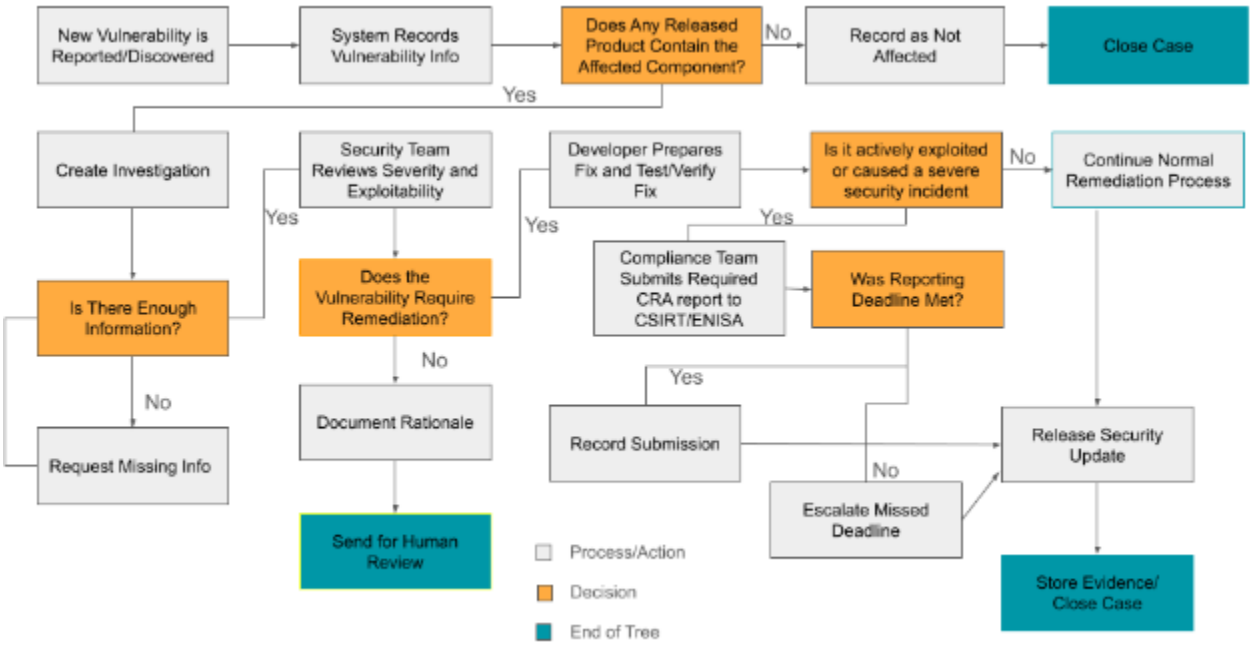
Create one or two diagrams showing the main participants, their responsibilities, and the major activities involved in addressing those obligations identified in Section 1.

Show decision points where a condition changes what happens next. State the condition as a question, label the possible outcomes, and show the action that follows each outcome. Include important exceptions—for example, missing information, a missed deadline, or a review finding that an obligation has not been met.

Show how the people, organizations, and external systems interact. Indicate which activities might benefit from software support and which responsibilities would remain outside that software's scope. This boundary is preliminary and does not bind your team to a particular system design.

Simplify deliberately: include the activities, relationships, and decisions needed to understand the challenge. You do not need to represent every detail of the regulation or every possible exception.

Simple labeled boxes, stick figures, and arrows are acceptable. Formal UML notation is not required, though you may use it – specifically, consider models such as use case diagrams and activity diagrams. You may supply whiteboard drawing, (vetted) LLM drawing of your model, draw.io or mermaid.js, etc.



Q2.2 Explanation (Richard)

(3–4 sentences) Explain what the model helped you understand, and connect the model to an obligation or the regulatory challenge. State any important assumptions about how responsibilities are assigned or activities are performed.

The model helped us understand that post-release vulnerability handling is not a single step, but a detailed process involving investigation, remediation, documentation, and possible regulatory reporting when necessary. It connects the CRA requirements for manufacturers to address vulnerabilities after release and report certain actively exploited vulnerabilities or severe security incidents. We assume that the security and compliance teams are responsible for making judgement-based decisions such as if the vulnerability requires remediation, while the software would mainly track information, deadlines, and evidence. The model also shows that software can support the workflow but human input is required in certain steps.

Q3 Existing-Solution Analysis (Ann)

Describe and compare two existing approaches that address (or could be used to address) aspects of this regulation. Look for existing:

- *Commercial products*
- *Open-source systems*
- *Organizational processes*
- *Standards and technical guidance*
- *Research prototypes*
- *APIs, libraries, and platforms that could form part of a solution*

For each of the two existing approaches, answer the following questions in 3-4 sentences.

1. ***Describe the solution**: Name the product, tool, or organizational process, briefly describe how it works, and cite your source.*
2. ***What does it already address?** Explain which part of your identified problem it handles and what it does well.*
3. ***How well does it fit your problem?** Describe a relevant limitation, an unmet need, or a question you would need to investigate before adopting it.*
4. ***What does this mean for your project?** Explain whether you would reuse it, integrate with it, or develop your own approach, and justify that direction.*

An existing solution may already address much of the problem. You do not need to demonstrate novelty; you need to explain what engineering work your team would undertake and why.

Q3.1 Solution 1

On GitHub there is a native feature called Dependabot that focuses on security vulnerabilities and outdated versions (source: https://github.com/dependabot). We want to focus on the post-release vulnerability, so this will help to address the security vulnerabilities in our products and fits in with what we want to do. However, Dependabot watches the current state of a repository rather than what was shipped to customers, so it has no record of which released versions contained a vulnerable component and no

concept of a reporting deadline. We could use this as a base function to build off for our project.

Q3.2 Solution 2

There exists a dependency-track which is an intelligent Component Analysis platform that will allow organizations to find and reduce risks within the software supply chain (source: https://dependencytrack.org/). This addresses the needs of the post-release by proactively reducing risk across an entire organization. Its limitation is that it has no concept of a regulatory obligation, deadline, or report. This will fit what we want to do for our project so we could use this method to build off for our project.

Q4 Candidate Problem Statements (Ann)

Describe two candidate problems for which software could help satisfy, operationalize, or provide evidence about the regulatory obligations you studied. (3-4 sentences each)

Identify the stakeholder and what they need to accomplish. Explain the current approach, its limitations, and what software could improve. Reference the relevant obligation from Section 1 and describe an observable outcome that would indicate success.

Q4.1 Candidate Problem 1

A manufacturer's security and compliance team must report actively exploited vulnerabilities at 24 hours, 72 hours, and again when a fix becomes available, as required by the Vulnerability and Incident Reporting obligation in Section 1.2. Today, they track this in a general ticket system with deadlines watched informally. However, that falls apart when an alert arrives over the weekend or the affected scope grows in the middle of an incident. Software could compute every deadline from one recorded awareness timestamp and draft each submission from the same case record; success would be observable if, given a replayed incident timeline, the system produces correct deadlines and complete submission drafts.

Q4.1 Candidate Problem 2

A manufacturer's security team must identify and document the components in products it has already released, as required by the Handling Vulnerabilities obligation in Section 1.2. Today, they answer this by scanning the current build and asking engineers to remember what shipped. However, the support period runs for years after a release, so a manufacturer can still owe a patch for a version nobody has a component inventory for. Software could maintain a robust index of components and support status from every shipped release; success would be observable if, given a vulnerability identifier, the system returns the complete set of affected shipped versions matching a manually verified answer.

Q5.1 Concept 1 (Quincy)

***Name and purpose (1–2 sentences)**: Name the candidate system and identify its intended users. Reference the problem and key obligation (from section 1.2) it addresses.*

The CRA Reporting Manager is a case-management system for product security and compliance teams at manufacturers subject to the CRA. It will track a vulnerability from the first signal through the final report in Article 14 and will also address Section 1.2 (Vulnerability and Incident Reporting) where a manufacturer must file within 24 hours, 72 hours, and again when the fix becomes available.

***Responsibilities and scope (2–3 sentences)**: Explain the candidate system's main responsibilities and the limits of its scope (Explain how it improves the current approach identified in Section 4).*

The system will accept vulnerability signals from multiple sources and will maintain a case record per vulnerability. It computes and monitors each reporting deadline from awareness timestamps to help keep manufacturers on track. For the three required reports, it drafts the content of the submissions, creates an advisory for affected users, and maintains an audit log of changes (for compliance evidence) but it will not perform the submission of the report as ENISA's reporting platform doesn't support API calls.

***Inputs and outputs (1–2 sentences)**: Describe what information it receives and what result it produces, using domain language.*

Inputs are vulnerability notifications (tickets, research reports, scanner findings, etc.), a list of affected products/versions, evidence the exploitation is real, and tracking the fix. Outputs are drafted for the early warning, detailed, and final reports.

***Evaluation (2 sentences)**: Give one example use case and its expected observable result. Explain how another team could check that result (Refine/Reuse the observable outcome from section 4).*

Given a mock situation, the system should be able to track multiple incidents independently of one another (the timeline for a report starting Friday should be treated differently from the timeline for a report starting Monday) and produce the needed documentation for each incident as needed. Another team could verify this by reproducing

the same situations, verifying the deadlines against hand calculations, and checking the generated documentation against the requirements outlined in law.

***Engineering risk (2–3 sentences)**: Identify the main technical uncertainty that could prevent the candidate system from working as intended. Explain its potential effect and one investigation or prototype that could reduce the uncertainty.*

The main uncertainty is whether the awareness timestamp can be established reliably. A manufacturer could get a ticket making them aware of the situation, but the ticket may not be processed by a support employee until later. We would reduce this uncertainty by writing a series of incident scenarios of varying complexities and testing whether a defensible awareness timestamp can be assigned in each.

***Potential reuse (1–2 sentences)**: Identify an existing tool, component, or service that could provide part of the system's functionality, and explain what you would reuse. If no suitable option has been identified, state that.*

We would use csaf-validator-lib (source: https://github.com/secvisogram/csaf-validator-lib) for validating generated advisories.

Q5.1 Concept 2 (Quincy)

***Name and purpose (1–2 sentences)**: Name the candidate system and identify its intended users. Reference the problem and key obligation (from section 1.2) it addresses.*

The Release Component Index is a lookup system for product security and release engineering teams at manufacturers subject to the CRA. It will track which shipped versions of a product contain a given component and which of those are still in their support period (this addresses Handling Vulnerabilities (Section 1.2) where a manufacturer must identify and document components throughout the support period).

***Responsibilities and scope (2–3 sentences)**: Explain the candidate system's main responsibilities and the limits of its scope (Explain how it improves the current approach identified in Section 4).*

The system will take in an SBOM for each shipped release and store it against the released artifact rather than a build. It records the support-period end date for each release and will track which shipped versions contain a certain component, or which release is still in support, rather than only what's in the current build. It will not generate SBOMs, scan source code, or decide the support period.

***Inputs and outputs (1–2 sentences)**: Describe what information it receives and what result it produces, using domain language.*

Inputs are SBOMs, release information (product, version, release date), support-period end dates, and a vulnerability or component to search for. Outputs are the list of affected shipped releases with their support status.

***Evaluation (2 sentences)**: Give one example use case and its expected observable result. Explain how another team could check that result (Refine/Reuse the observable outcome from section 4).*

If provided with a set of test releases with known contents, the system should return exactly what in-support releases are affected by a given CVE. Another team could verify this by building their own test set with a known answer and checking the list that the system returns.

***Engineering risk (2–3 sentences)**: Identify the main technical uncertainty that could prevent the candidate system from working as intended. Explain its potential effect and one investigation or prototype that could reduce the uncertainty.*

The main uncertainty is whether components can be matched reliably across formats. The same library can show up under different names or version conventions. We would reduce this uncertainty by prototyping the matching process against a small set of real SBOMs and measuring how often it matches correctly.

***Potential reuse (1–2 sentences)**: Identify an existing tool, component, or service that could provide part of the system's functionality, and explain what you would reuse. If no suitable option has been identified, state that.*

We might use OWASP Dependency-Track (source: https://dependencytrack.org/) for SBOM ingestion and component matching, adding the release and support-period tracking on top.

Q6 Pod Findings (Quincy)

After your team completes Sections 1–5, meet with the other teams in your pod. Record the meeting date and participating team and attendees, then write a 150–200-word summary in two paragraphs. Each pod-team should write its own independent summary.

TEAM 17 AND TEAM 21 MET ASYNCHRONOUSLY ON SEPTEMBER 13, 2026.

***Paragraph 1**: Summarize the most consequential finding from the pod discussion and explain how it changed or clarified your team's analysis.*

***Paragraph 2**: Describe an important disagreement or unresolved question. Explain what further investigation is needed to help resolve this issue, and the alternatives worth considering in Phase 2. If there was no disagreement, that would be a miracle. However, in that case, identify a remaining uncertainty.*

A big finding between both teams is that we each believe that we should focus on using the SBOM in conjunction with a vulnerability to find affected product/version releases. We are both on the same page that OWASP Dependency-Track is a good candidate to be reused in our project. We also both identified the same risk of SBOM component identifiers not matching the identifiers used in vulnerability databases. This reinforces the idea that detection of a vulnerability is not where we should spend our time.

For some potential solutions, both teams are on slightly different pages on how that would work. Team 21 focused more on how reporting in to the manufacturer would work whereas we focused on smoothing out the reporting out process. Building from that idea, the actual understanding of "becoming aware" differs between our teams. Team 21 understood that phrase as if an exploitation has been reported and it hasn't been verified or used, then a report out is not necessary at that time. We took "becoming aware" as soon as there's a ticket/email/message about a vulnerability and someone sees it, then the clock starts. We'll have to do some more research in Phase 2 to see which understanding makes more sense, and in turn, which approach to the solution works best.